UNIVERSITÀ DEGLI STUDI FIRENZE
DINFO
Dipartimento di Ingegneria dell'Informazione

CUDA-based Parallelization of Kernel Image Processing

Chiara Gori

Problem Introduction

- Problem: convolution on RGB images with a Gaussian kernel → blurring / noise reduction

- General expression of a convolution:

$$g_{x,y} = \omega * f_{x,y} = \sum_{i=a}^{a} \sum_{j=b}^{b} \omega_{i,j} f_{x-i,y-j}$$

where $g(x,y)$ is the filtered image, $f(x,y)$ is the original image, ω is the kernel filter, and every element of the kernel is considered by $-a \leq i \leq a$ and $-b \leq j \leq b$

Problem Introduction

- A Gaussian kernel (or filter) has weights based on:

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

- Common example of Gaussian kernel:

$$K = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Problem Introduction

- Computationally challenging:
  - For every pixel, for every channel, k^2 multiply-accumulate ops

    → total cost: $W \times H \times k^2 \times C$

- Natural candidate for parallelization:
  - Every output pixel is independent from the others

Approach

- Pixel values are stored as uint8_t
  - Each RGB channel takes integer values in range [0, 255]
- Chosen data layout:
  - Conversion to SoA from AoS (motivated by memory coalescing on the GPU)

$$[R_0, R_1, \ldots, G_0, G_1, \ldots, B_0, B_1, \ldots]$$

- Pre-computed zero-padding
  - Removes the need for boundary checks inside the CUDA kernel
- CPU baseline:
  - Single threaded + loops over channels → rows → columns → k×k window + clamped to [0, 255]

Parallelization strategies

- 5 variants:

Variant	Channels	Coefficients	Key idea
1ChNoConst	sequential loop	global memory	GPU baseline
1ChConst	sequential loop	constant memory	broadcast cache
3ChGrid	3D grid (z=channel)	constant memory	channel parallelism (TLP)
3ChNoGrid	3 accumulators	constant memory	ILP
3Ch3Arrays	3 separate arrays	constant memory	explicit coalescing

Parallelization Strategies

- Constant memory + broadcast
  - Every thread in a warp reads the same coefficient
    → a single memory transaction instead of many
- ILP vs TLP:
  - Independent per-channel accumulators allow to overlap the multiply-accumulate chains
    → more effective than thread-level parallelism across channels
- 3 block sizes:
  - 8×8, 16×16, 32×32

Experimental Setup

- Dataset:
  - Five images at five increasing resolutions:
    - 360p (640×360)
    - 720p (1280×720)
    - 1080p (1920×1080)
    - 2K (2560×1440)
    - 4K (3840×2160)

- 5 variants × 3 block sizes, two experiments:
  - Fixed kernel (11×11), resolution from 360p to 4K
  - Fixed resolution (1080p), kernel size from 3×3 to 33×33

Experimental Setup

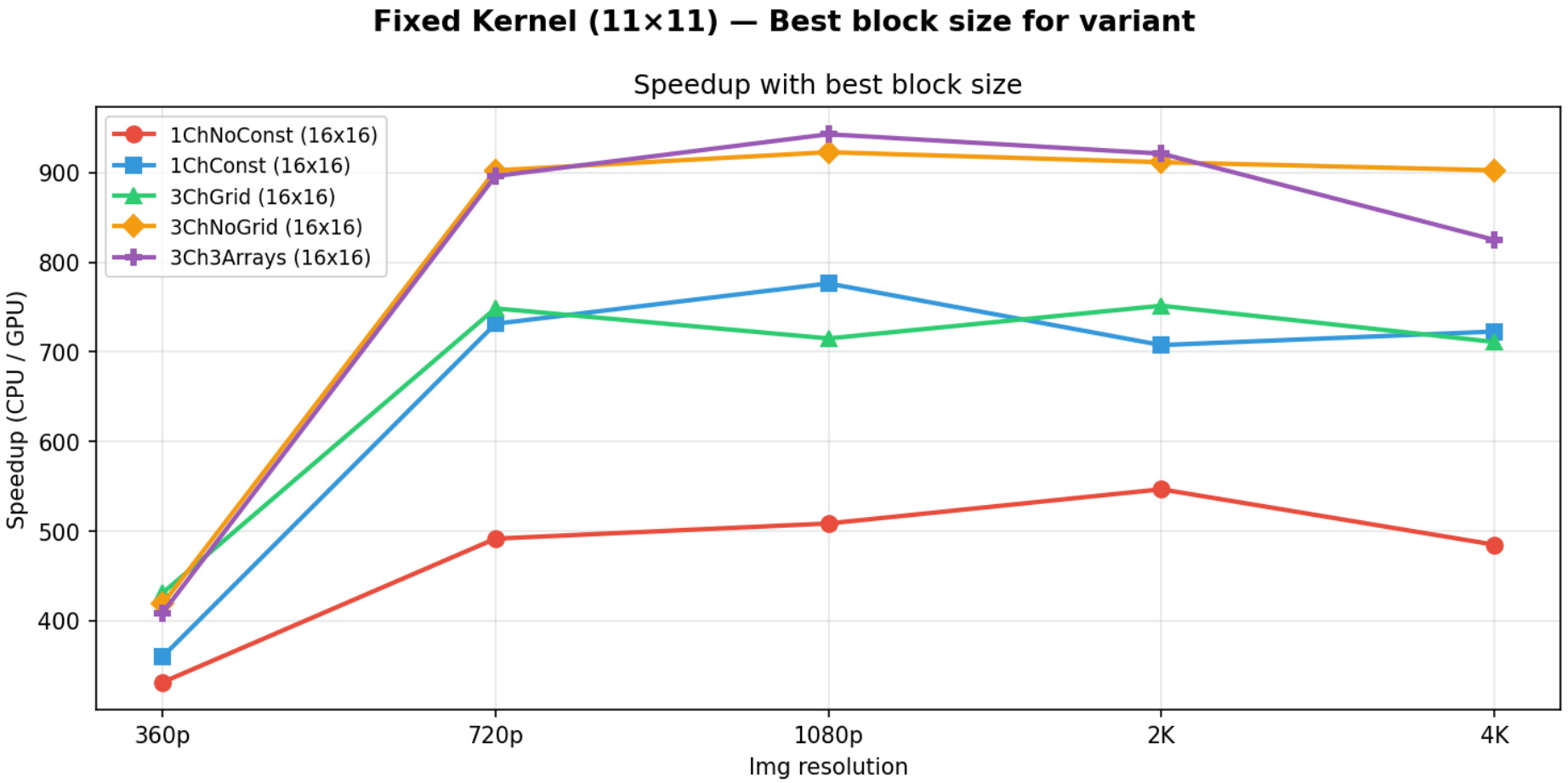
- Methodology:
  - 50 runs for configuration
    - First 2 runs discarded as warm-up
    - Average over the remaining 48
- Validation:
  - Pixel-by-pixel comparison CPU vs GPU
  - ±1 pixel tolerance
- GPU: NVIDIA GeForce RTX 3070 (Ampere architecture, Compute Capability 8.6, 5888 CUDA cores, 8GB VRAM)
- SW: Ubuntu 24.04 LTS under WSL2, hosted on Windows 11 + code compiled with nvcc

Experimental Setup

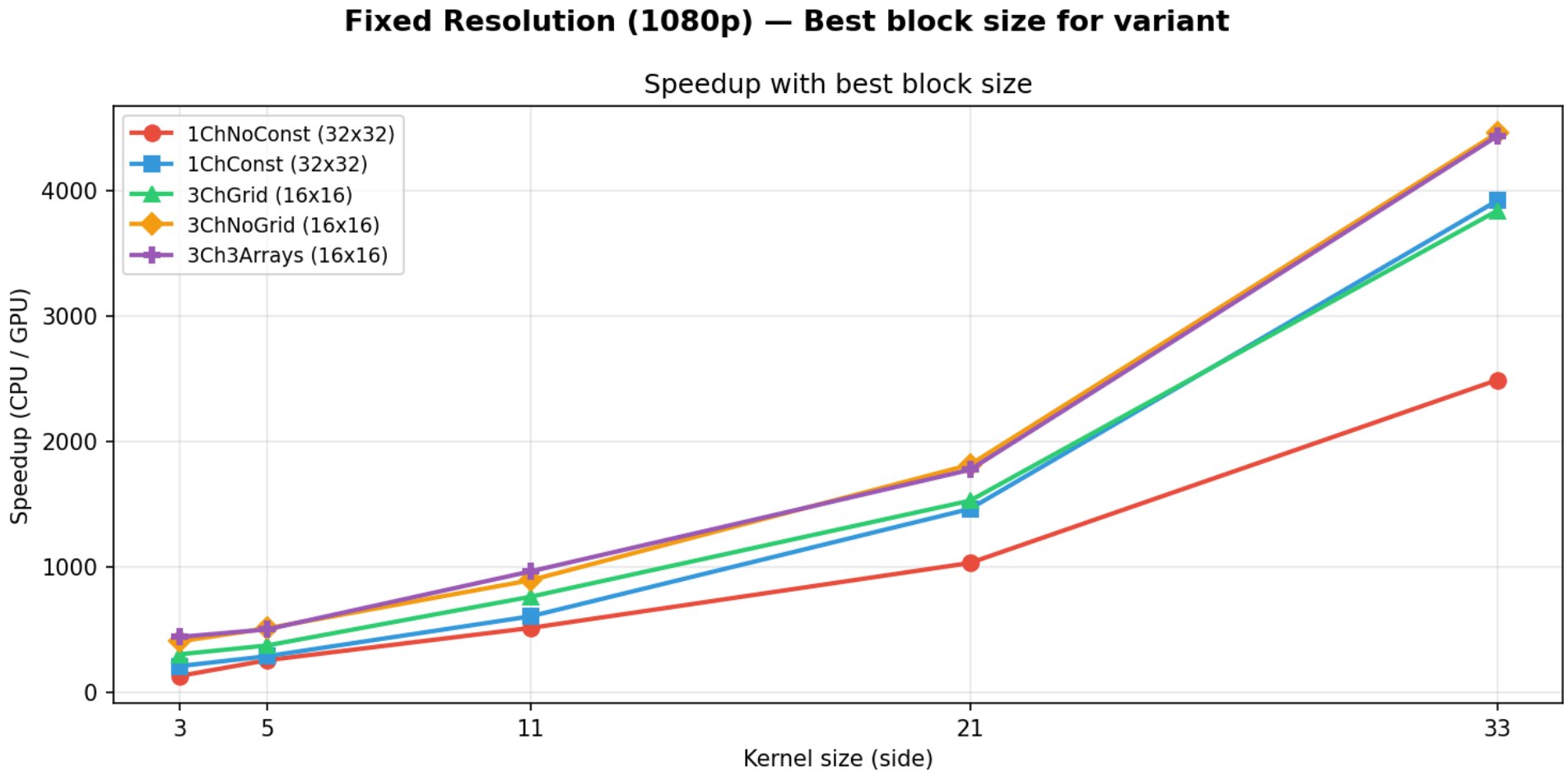
- Main metric:
  - **speedup** = sequential time / parallel time

- 3 timers:
  - CPU baseline: std::chrono::high resolution clock
  - Two timers for GPU (CUDA Events):
    - Kernel-only computation
    - Total (includes memory allocation and PCIe transfer)

Results – Fixed Kernel



Results – Fixed Resolution

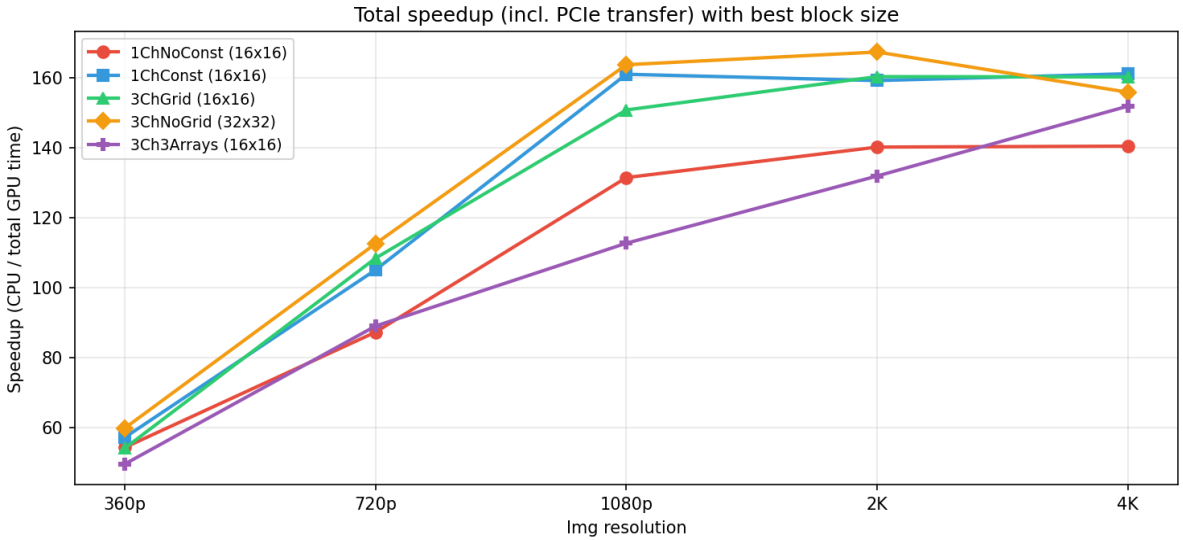


Best Configuration Achieved

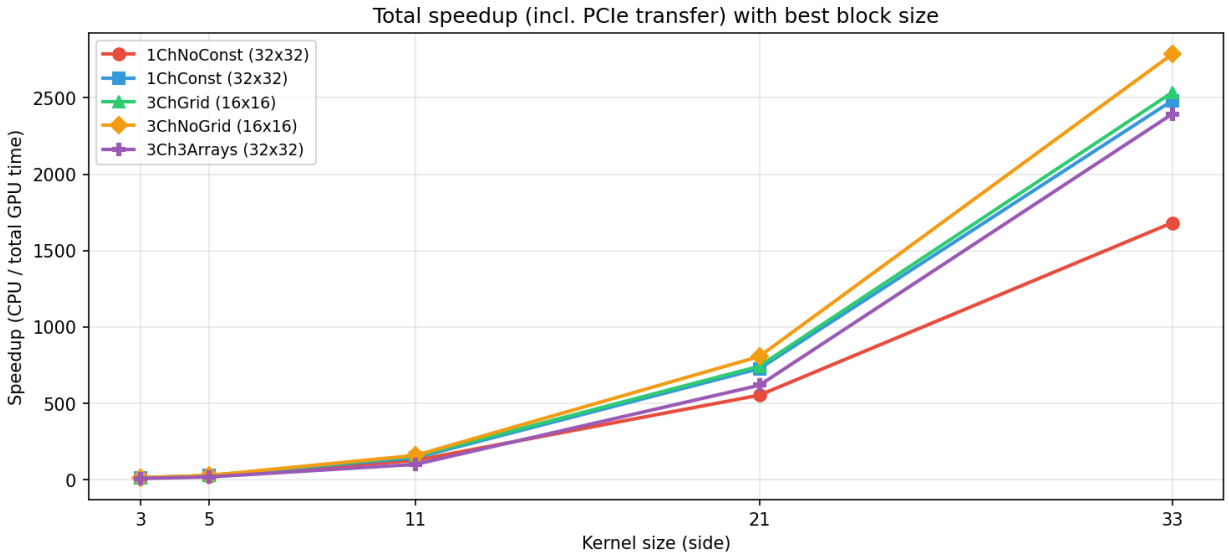
- Peak speedup:
  - 4455x (kernel-only) at k = 33×33 + 1080p
- Overall best variant:
  - 3ChNoGrid (constant memory + ILP + single continuous buffer)
- Overall best block size:
  - 16×16
  - 100% occupancy on the RTX 3070
    - 6 blocks/SM
    - 48 warps schedulable independently

Results including PCIe transfer time

Fixed Kernel (11×11) — Total speedup with best block size



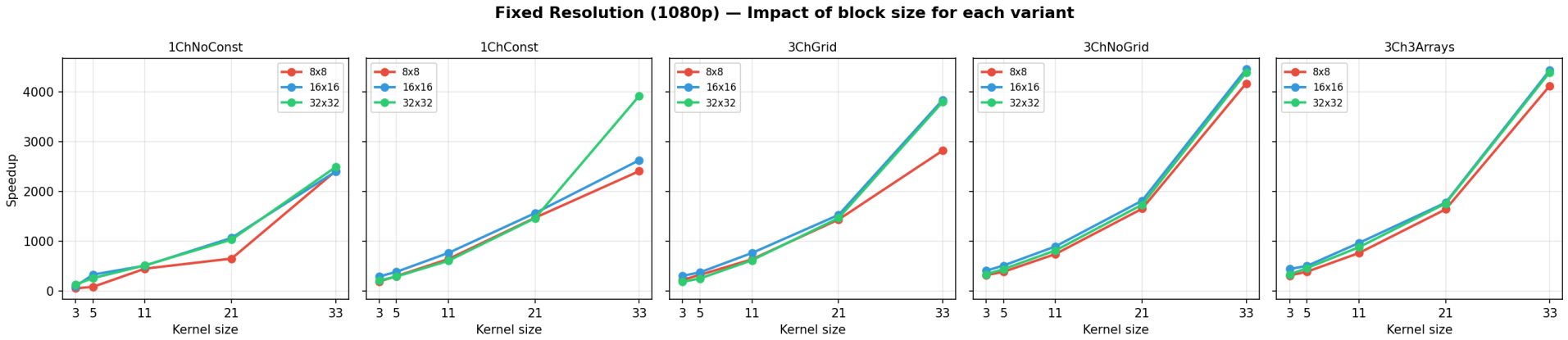
Fixed Resolution (1080p) — Total speedup with best block size



Conclusions

- Very different achieved speedups
  - Much higher for kernel-only timer
  - Lower for end-to-end timer
- Overall best variant with both timers:
  - 3ChNoGrid (constant memory + ILP + single continuous buffer)
- 3Ch3Arrays performs nice for kernel-only but much worse for end-to-end due to fixed overhead
  - 6 separate memory allocation calls

Extra slide


Fixed Resolution (1080p) — Impact of block size for each variant
1ChNoConst
1ChConst
3ChGrid
3ChNoGrid
3Ch3Arrays
8x8
16x16
32x32
Speedup
0
1000
2000
3000
4000
3
5
11
21
33
Kernel size